

Gemma-GLM v2.1

Geometric Language Machine — Lean4-Verified

Gemma-GLM is a deterministic, rule-based companion for any LLM. It combines a 114-entry typed Concept Relation Graph (CRG) with Golay [24,12,8] error-correcting code geometry and Leech lattice coherence scoring to provide syllogistic inference, contradiction detection, and principled candidate selection. v2.1 refines the mathematics using Lean4-verified theorems: the MOG-aware TAX distinguishes lawful from unlawful bit patterns, the calibrated coherence budget makes all four regimes meaningful, and the hallucination-block claim is honestly reframed as a three-tier deference policy. Zero trainable parameters; Python stdlib only.

1. What it is

Gemma-GLM is not a neural language model. It is a geometric reasoning layer that wraps around an existing LLM to add deterministic verification. The system encodes tokens as 24-bit vectors using Gray code (preserving topological adjacency), snaps them to the nearest Golay codeword for noise robustness, and scores them using a five-shell Non-Random Coherence Index (NRCI) derived from the Leech lattice.

On top of this substrate sits a typed Concept Relation Graph with 14 edge labels (`is_a`, `has_property`, `depends_on`, `contradicts`, `contains`, `transforms`, and others). The CRG enables syllogistic inference ("Is a photon a boson?" → direct edge), inherited property lookup (photon inherits massless via `is_a` boson), and contradiction detection (photon contradicts fermion, inherited through the `is_a` chain).

The pipeline uses CRG graph distance — not hash proximity — to select candidates, achieving 90% gold-pick rate on a 30-prompt benchmark (matching Levenshtein, versus 40% for hash-based selection). A $2\Delta v$ differential transition engine detects invariant transformation rules across input/output pairs, and HexColour addressing provides a stable lattice signature for analogical reasoning.

2. What changed in v2.1

v2.1 incorporates refinements derived from Lean4-verified theorems. Each change is backed by a machine-checked proof and verified empirically by 47 property tests.

MOG-aware TAX (Lean4 §14 A)

The symmetry tax now includes a syndrome penalty: $\text{TAX_MOG}(\mathbf{v}) = \text{HW}(\mathbf{v}) \cdot Q + \text{syndromePenalty}(\mathbf{v})$, where $\text{syndromePenalty}(\mathbf{v}) = \text{HW}(\text{leader}(\sigma(\mathbf{v}))) \cdot Q$. This distinguishes lawful codewords (penalty = 0) from random bit patterns of the same weight (penalty > 0) — something v2.0's coarse syndrome-weight heuristic could not do.

Calibrated coherence budget (Lean4 §8)

The budget B is now $8Q$ (the cheapest protected distinction) instead of the stipulated 10. For codewords, NRCI simplifies to $\frac{8}{8+w}$ where w is the Hamming weight. The octad (weight-8 codeword) sits exactly at $\text{NRCI} = 0.500$, the Coherent boundary. All four regimes — OnBit, Coherent, Transitional, Subcoherent — are now meaningfully separated.

Honest unknown-word policy

v2.0 claimed "100% hallucination block," which was misleading (it was a low baseline score, not a real block). v2.1 honestly classifies unknown words into four tiers: incoherent (score 0.00, geometrically far from any codeword), lawful_unknown (0.15, snaps to a codeword but not in CRG), analogue (0.30, geometrically near a zone word), and unknown (0.05, no geometric claim).

CRG distance memoization

Distance queries are now memoized with automatic cache invalidation on graph mutation. Repeated queries drop from 2541 μ s to approximately 1 μ s (98% hit rate), making the pipeline 3.4 $\times$ faster overall than v2.0.

Y correction

The claim "Y is the minimum cost of reading" is false — the Lean4 proof shows the read operator $Y[\Pi]$ has no positive lower bound. v2.1 honestly states that $Y = 1/(\pi + 2/\pi)$ is the read cost at the stipulated loop-check $\Pi = \pi$, not an extremum. The Q_{le_tax} theorem (Q is the minimum tax) is unaffected.

3. How to use it

Quick start

```
# Run the v2.1 self-test (19 checks + 18 v1 compat checks)
python3 gemma_glm_v2.py --test
```

```
# Interactive REPL
python3 gemma_glm_v2.py
```

```
# Run all 227 tests
python3 -m pytest tests/ -v
```

CLI commands

```
python3 gemma_glm_v2.py --encode photon          # Gray-code encoding
python3 gemma_glm_v2.py --ncrci photon          # v1 vs v2.1 NRCI
python3 gemma_glm_v2.py --shells photon         # Full 5-shell breakdown
python3 gemma_glm_v2.py --infer "Is a photon a boson?"
python3 gemma_glm_v2.py --hexcolour photon      # Lattice address
python3 gemma_glm_v2.py --compare photon light  # Hamming + HexColour distance
python3 gemma_glm_v2.py --pipeline "explain photon"
python3 gemma_glm_v2.py --bench                 # Run benchmarks
```

Python API

```
from llm_glm.pipeline_v2 import create_v2_pipeline

pipe = create_v2_pipeline()
r = pipe.process("explain photon", ["frequency", "kitchen", "boson"])
print(r.selected)           # "frequency" or "boson"
print(r.candidate_scores)   # Full scoring with deference_tier
print(r.inference)          # Syllogistic inference result
print(r.zone_hexcolour)     # Lattice address
```

Individual modules

```
from llm_glm.gray_encoder import encode_token          # Gray code
from llm_glm.refined_ncrci import RefinedNRCI, classify_regime # 5-shell NRCI
from llm_glm.differential import compute_2dv, analyze_transitions # 2Δv
from llm_glm.expanded_crg import build_expanded_crg    # Typed CRG
from llm_glm.reasoning import syllogistic_inference, score_candidates
```

```
from llm_glm.hf_logits_processor_v2 import create_glm_processor_v2 # HuggingFace
```

HuggingFace integration

```
from llm_glm.hf_logits_processor_v2 import create_glm_processor_v2

processor = create_glm_processor_v2(tokenizer=tokenizer, bias_strength=1.0)
outputs = model.generate(input_ids, logits_processor=[processor])
```

v1 vs v2.1

Both versions coexist. Use **python3 gemma_glm.py** for v1 (original 18-check self-test, hash-based pipeline) or **python3 gemma_glm_v2.py** for v2.1 (Lean4-verified, CRG-distance pipeline, honest unknown-word policy). v1 is 100% preserved — all 108 original tests still pass.

4. Benchmark results

Pipeline selection (30 test prompts, 4 gold + 4 distractor candidates)

Method	Gold picks	Gold rate
v1 Pipeline (SHA-256 hash proximity)	12/30	40.00%
v2.1 Pipeline (CRG distance)	27/30	90.00%
Random choice	15/30	50.00%
Levenshtein (mean)	27/30	90.00%

Latency (Python 3.12, CPU only)

Operation	v1	v2.0	v2.1
Pipeline.process (5 candidates)	615 µs	1754 µs	512 µs
CRG distance (cached)	248 µs	2541 µs	~1 µs
Token encoding	2.3 µs	9.3 µs	9.3 µs
NRCI computation	8.6 µs	24.4 µs	24.4 µs

Lean4 theorem verification (47 property tests)

Theorem	Statement	Verified
Q_le_tax	$TAX(v) \geq Q$ for every nonzero v	500 trials
tax_eq_Q_iff	$TAX = Q$ iff single ± 1 coord	Yes
protection_costs_eight_quanta	Cheapest protected = $8Q$ (octad)	759 octads
syndromePenalty_eq_zero_iff	Penalty = 0 iff lawful	400 trials
syndromePenalty_le	Penalty $\leq 4Q$ (covering radius)	1000 trials
taxMOG_eq_bitTax_iff	$TAX_MOG = TAX$ iff lawful	400 trials
history_additive	$\sigma(a \oplus b) = \sigma(a) \oplus \sigma(b)$	500 trials
same_history_iff	$\sigma(a) = \sigma(b)$ iff $a \oplus b$ lawful	500 trials

Unknown-word deference (40 unknown words, honest reporting)

Tier	Score	Meaning	Count
incoherent	0.00	Geometrically far from any codeword	0
lawful_unknown	0.15	Snaps to codeword but not in CRG	29
analogue	0.30	Geometrically near a zone word	0
unknown	0.05	No geometric claim	11

5. What v2.1 claims (and does not claim)

Claims made

- Pipeline selection achieves 90% gold-pick rate, matching Levenshtein, driven by CRG graph distance.
- The MOG-aware TAX distinguishes lawful from unlawful same-weight patterns — verified by Lean4.
- The calibrated budget $B = 8Q$ gives meaningful regime separation (octad at NRCI = 0.500).
- Eight Lean4 theorems are verified empirically across 500–1000 random trials each.
- CRG distance memoization gives 2500× speedup on repeated queries (98% hit rate).

Claims NOT made

- Not a hallucination blocker. The system ranks candidates; it does not refuse to generate. "Deference" is the honest word.
- Not a neural LLM. Gemma-GLM is a deterministic, rule-based wrapper. The intelligence is in the CRG and the Golay/Leech geometry.
- Y is not derived. $Y = 1/(\pi + 2/\pi)$ is the read operator at the stipulated $\Pi = \pi$, not an extremum. The Lean4 document proves the operator has no positive lower bound.
- Physics formulas are fitted, not predicted. The constants (169, 24, $1/3$) were tuned to match known values.
- NRCI discrimination AUC is 0.79 (down from v2.0's 0.84). This is a deliberate trade: the calibrated budget gives meaningful regimes but compresses word-level discrimination. Both modes are available.

6. File structure

```
Gemma-GLM/
├─ gemma_glm.py           # v1 (preserved)
├─ gemma_glm_v2.py        # v2.1 CLI + self-test
├─ ubp_unified_v5.py       # UBP engine (shared)
├─ llm_glm/
│   ├─ refined_nrci.py     # 5-shell NRCI + MOG-aware TAX + Lean4 theorems
│   ├─ gray_encoder.py     # Gray code encoding
│   ├─ differential.py     #  $2\Delta v$  transitions + HexColour
│   ├─ expanded_crg.py     # Typed CRG (14 labels) + memoized distance
│   ├─ reasoning.py        # Syllogistic inference + honest 3-tier policy
│   ├─ pipeline_v2.py      # v2.1 pipeline
│   ├─ hf_logits_processor_v2.py # HuggingFace integration
│   └─ agent_v2.py         # Interactive REPL
├─ tests/
│   ├─ test_gemma_glm.py   # 108 v1 tests
│   └─ test_gemma_glm_v2.py # 72 v2 tests
```

```
|   └─ test_lean4_properties.py # 47 Lean4 theorem tests
└─ V2_README.md                # Full documentation
```

Version 2.1.0 · 10 August 2026 · Author: E.R.A. Craig (DigitalEuan) · Python ≥ 3.10, stdlib only · 227 tests passing